# LES SYSTÈMES D'EXPLOITATION MULTIPROCESSEURS

*Synthèse simplifiée : Fondements, Noyau, Ordonnancement, Concurrence et Accélérateurs (GPU/IA)*

Document de cours condensé avec références bibliographiques et exemples simples

---

## Introduction : L'impasse physique et le virage du multicœur

Pendant près de trente ans, la vitesse des logiciels augmentait toute seule à chaque nouveau processeur. On achetait une nouvelle machine, et le même logiciel tournait deux fois plus vite sans qu'un développeur ne touche au code. Herb Sutter a résumé ça sous une formule célèbre : la fin du *"Free Lunch"* **[Sutter, 2005]**. Ce miracle reposait sur deux piliers : la loi de Moore et l'échelle de Dennard.

### 1. L'âge d'or du monocœur

En 1965, Gordon Moore constate que le nombre de transistors par puce double environ tous les deux ans **[Moore, 1965]**. De son côté, Robert Dennard démontre en 1974 qu'en rétrécissant un transistor, sa consommation baisse proportionnellement. Résultat : la densité d'énergie reste constante. On pouvait donc augmenter la fréquence d'horloge de la puce sans la transformer en grille-pain **[Dennard et al., 1974]**.

> **ANALOGIE : LA VOITURE SUR VOIE UNIQUE**
> C'est comme si on améliorait une voiture sur une route à une voie : chaque année, le moteur tourne deux fois plus vite. Le trajet prend deux fois moins de temps sans changer la façon de conduire.

### 2. La rupture physique : les trois murs (vers 2005)

Au milieu des années 2000, les transistors sont devenus tellement petits que les fuites d'électricité ont fait grimper la température. L'échelle de Dennard s'est effondrée contre trois murs physiques **[Silberschatz et al., 2018]** :

- **Le mur de la puissance :** La puce chauffe trop et devient impossible à refroidir avec un ventilateur classique.
- **Le mur de la fréquence :** Bloqué net autour de 3 à 4 GHz. Monter plus haut risquerait de griller le silicium.
- **Le mur de la mémoire :** Le processeur va trop vite par rapport à la mémoire RAM et passe son temps à l'attendre.

### 3. Le virage du multicœur

Les besoins en calcul n'ont pas diminué pour autant. Les fabricants ont donc changé de stratégie : au lieu de fabriquer un seul cœur ultra-rapide, ils ont placé **plusieurs cœurs** sur la même puce. Mais attention, un système d'exploitation conçu pour un seul cœur ne sait pas utiliser cette puissance. C'est à l'OS de répartir les tâches et de gérer les accès simultanés sans pagaille **[Tanenbaum & Bos, 2015]**.

> **ANALOGIE : LE CONVOI DE CAMIONS**
> Quand on ne peut plus fabriquer une voiture de course roulant à 1000 km/h, on envoie dix camions à 100 km/h. On transporte plus de marchandises, mais il faut un bon régulateur pour éviter les collisions.

### 4. Définition

Un **système d'exploitation multiprocesseur** répartit les programmes entre plusieurs cœurs, gère la mémoire partagée et coordonne les accès aux ressources pour éviter que deux traitements ne se marchent sur les pieds [Stallings, 2018].

> *Maintenant que l'OS doit tourner lui-même sur plusieurs cœurs, voyons comment son propre noyau protège ses données.*

## Partie 1 : Les modèles d'OS et l'organisation du noyau

### 1.1 Définition du noyau

Le **noyau (kernel)** est la partie centrale et privilégiée de l'OS. C'est lui qui contrôle directement la mémoire, le matériel et le lancement des programmes [Love, 2010].

### 1.2 Le modèle asymétrique (AMP)

Dans le modèle AMP, un processeur **maître** exécute tout le noyau. Les autres processeurs, dits **esclaves**, font tourner les applications et envoient leurs demandes au maître [Silberschatz et al., 2018].

- *Avantage :* Très simple à écrire, aucun risque de conflit dans le noyau.
- *Inconvénient :* Le maître sature vite et si le maître tombe en panne, toute la machine s'arrête.

> **ANALOGIE : LA CUISINE AVEC UN SEUL CHEF**
> Plusieurs commis préparent les plats, mais un seul chef garde les clés du garde-manger. Dès qu'un commis a besoin d'un ingrédient, il doit faire la queue devant le chef.

### 1.3 Le modèle symétrique (SMP)

Dans le modèle SMP, tous les processeurs sont égaux et chacun peut exécuter le noyau [Tanenbaum & Bos, 2015]. C'est le modèle utilisé par Linux, Windows et macOS.

- *Avantages :* La charge est bien répartie et le système continue de tourner si un cœur flanche.
- *Défi :* Si deux cœurs modifient en même temps la liste des programmes en mémoire, ils risquent de la corrompre.

### 1.4 Comparaison rapide AMP / SMP

| Critère | Modèle Asymétrique (AMP) | Modèle Symétrique (SMP) |
|---|---|---|
| **Exécution du noyau** | Processeur maître uniquement. | Tous les processeurs. |
| **Risque de bouchon** | Élevé (le maître sature vite). | Faible (réparti selon les verrous). |
| **Passage à l'échelle** | Limité à quelques cœurs. | Très bon (dizaines de cœurs). |

### 1.5 Protéger le noyau en SMP

- **Le verrou global (Big Kernel Lock - BKL) :** Un gros cadenas unique pour tout le noyau. Un seul cœur peut entrer dans le noyau à la fois **[Bovet & Cesati, 2005]**. (Définitivement retiré du noyau Linux en 2011 avec la version 2.6.39).
- **Les verrous fins :** Un cadenas séparé pour chaque petite liste de données. Ça demande plus de prudence, mais le gain de vitesse est énorme **[Love, 2010]**.

> *Le noyau étant sécurisé, comment l'OS choisit-il quel cœur va exécuter quel programme ?*

## Partie 2 : L'ordonnancement des processus

### 2.1 Organisation des files d'attente

- **File globale unique :** Tous les cœurs piochaient dans la même liste. C'est très équitable, mais les cœurs perdent du temps à se disputer le cadenas de la file **[Silberschatz et al., 2018]**.
- **Files locales par cœur :** Chaque cœur a sa propre liste. Pas de dispute sur le cadenas, mais un cœur peut se retrouver au chômage pendant que son voisin déborde de travail.

> **ANALOGIE : LES CAISSES DU MAGASIN**
> Une file unique (comme à la banque) garantit le passage dans l'ordre, mais crée un bouchon. Une file par caisse (au supermarché) va vite, sauf si quelqu'un devant vous a un caddie plein à craquer.

### 2.2 Équilibrage de charge et affinité

Pour régler le problème des files locales, l'OS utilise le **Work Stealing** : un cœur qui a fini son travail va piquer des tâches dans la file de son voisin **[Tanenbaum & Bos, 2015]**. En même temps, l'OS essaie de garder un programme sur le même cœur (**affinité processeur**) pour ne pas vider la mémoire cache L1/L2 qui contient déjà ses données.

### 2.3 Les machines NUMA

Sur les grands serveurs à plusieurs puces, la mémoire RAM est découpée par groupe de processeurs : c'est l'architecture **NUMA (Non-Uniform Memory Access)**. Accéder à sa mémoire locale prend quelques nanosecondes, alors qu'aller chercher la mémoire d'un autre processeur est beaucoup plus long. L'ordonnanceur veille donc à placer le programme et ses données sur le même nœud physique **[Bovet & Cesati, 2005]**.

> *Faire travailler plusieurs cœurs en parallèle crée un vrai risque : les erreurs de concurrence.*

## Partie 3 : Synchronisation, concurrence et pièges logiciels

### 3.1 Erreurs de concurrence et opérations atomiques

Une **condition de concurrence (race condition)** arrive quand deux cœurs lisent et écrivent sur la même donnée au même instant, écrasant le travail de l'autre **[Stallings, 2018]**. Couper les interruptions sur un seul cœur ne sert à rien en SMP. On s'appuie sur des instructions matérielles **atomiques** (comme `Compare-and-Swap`) qui bloquent la ligne mémoire le temps d'une modification.

### 3.2 Spinlock ou Mutex ?

- **Spinlock :** Le cœur tourne en boucle en attendant que la place se libère. Très rapide si l'attente duren deux microsecondes, mais ça brûle de l'électricité inutilement si ça s'allonge **[Love, 2010]**.
- **Mutex :** Le programme s'endort et laisse sa place à un autre. C'est plus sobre, mais réveiller le programme demande un changement de contexte coûteux.

### 3.3 Les bugs classiques

- **Deadlock (Interblocage) :** Deux programmes se bloquent mutuellement en attendant une ressource tenue par l'autre. C'est l'analogie du carrefour où quatre voitures arrivent en même temps et se cèdent la priorité à droite. (Voir conditions de Coffman **[Coffman et al., 1971]**).
- **False Sharing (Faux partage) :** Deux variables différentes partagent la même ligne de cache de 64 octets. Les cœurs se renvoient le bloc mémoire en permanence, effondrant la vitesse du système sans qu'il n'y ait la moindre erreur de code **[Tanenbaum & Bos, 2015]**.

> *Pourquoi ne peut-on pas accélérer un programme à l'infini en ajoutant des cœurs ?*

## Partie 4 : Limites théoriques et évolution

### 4.1 La loi d'Amdahl

La **loi d'Amdahl** (1967) montre que l'accélération d'un logiciel est limitée par sa partie séquentielle (1 - p), c'est-à-dire les lignes de code qu'on ne peut pas découper **[Amdahl, 1967]** :

```
Accélération = 1 / [ (1 - p) + (p / n) ]
```

S'il reste seulement 5 % de code non parallélisable (1 - p = 0,05), l'accélération maximale théorique ne dépassera jamais **20**, même avec un million de cœurs.

### 4.2 L'asymétrie moderne (big.LITTLE)

Pour économiser la batterie des smartphones et éviter la surchauffe des PC, les puces actuelles mélangent de gros cœurs puissants (P-cores) et de petits cœurs économes (E-cores) **[Silberschatz et al., 2018]**. L'OS doit envoyer les jeux lourds sur les gros cœurs et la musique de fond sur les petits cœurs.

> *Aujourd'hui, l'histoire va encore plus loin : les processeurs collaborent avec des puces spécialisées.*

## Partie 5 : L'ère des accélérateurs spécialisés (GPU et Puces IA)

### 5.1 Le GPU (Graphics Processing Unit)

Un GPU contient des centaines ou des milliers de cœurs très simples qui font exactement la même opération sur des blocs de données massifs (mode SIMD/SIMT) [Hennessy & Patterson, 2017]. Conçu pour la 3D, il est devenu incontournable pour les calculs scientifiques et l'intelligence artificielle.

**Rôle de l'OS :** L'OS utilise un pilote spécialisé pour envoyer les données au GPU, gérer sa mémoire et partager son accès entre plusieurs applications (votre jeu et votre navigateur). Sur un ordinateur fixe, le GPU a sa propre mémoire VRAM séparée, alors que sur un téléphone portable, il partage la mémoire RAM du système avec le CPU.

### 5.2 Les puces d'IA (NPU, TPU)

Les **NPU (Neural Processing Unit)** et **TPU (Tensor Processing Unit)** sont des circuits intégrés dessinés spécifiquement pour accélérer les réseaux de neurones (les multiplications de matrices) [Hennessy & Patterson, 2017]. Ils consomment très peu d'énergie.

**Exemples concrets :** Le Neural Engine des iPhone d'Apple, les processeurs Tensor des smartphones Pixel, ou les NPU Qualcomm des nouveaux PC. Ces puces gèrent la reconnaissance vocale, la retouche photo automatique et la traduction en direct sur votre appareil, sans envoyer vos données sur un serveur externe.

**Rôle de l'OS :** Choisir si un calcul doit aller sur le CPU, le GPU ou le NPU, tout en fournissant des outils simples aux développeurs (comme Core ML chez Apple).

### 5.3 Le smartphone : Un SoC où tout cohabite

Sur un téléphone, l'intégralité du matériel est regroupée sur une seule puce appelée **SoC (System on Chip)** : cœurs CPU big.LITTLE, GPU, NPU, modem et traitement d'image [Tanenbaum & Bos, 2015]. L'OS mobile (Android ou iOS) doit faire tourner tout ce monde en pensant à la batterie et à la chaleur avant la vitesse pure.

### 5.4 Ce qui change pour l'OS

- **Ordonnancement :** L'OS ne distribue plus seulement du temps CPU, mais envoie des blocs de travail vers des composants très différents.
- **Mémoire & Énergie :** Éviter de recopier les données d'une puce à l'autre et éteindre les zones inactives du composant pour sauver la batterie.
- **Firmware :** Un petit programme interne (firmware) tourne directement sur le GPU ou le NPU. L'OS ne contrôle pas tout dans le détail, il dialogue avec le composant via des commandes.

## Conclusion

L'évolution des systèmes d'exploitation découle directement des limites physiques du matériel. Pour surmonter les blocages de fréquence et de température, l'OS a dû s'adapter à chaque étape : du modèle asymétrique au SMP, du verrou unique aux verrous fins, et de l'ordonnancement NUMA jusqu'à la gestion des smartphones hybrides actuels. L'OS moderne est désormais le chef d'orchestre d'une mosaïque de puces spécialisées.

## Bibliographie et Références Académiques


1. **Amdahl, G. M. (1967).** *Validity of the single processor approach to achieving large scale computing capabilities.* AFIPS Conference Proceedings, 30, 483-485.
2. **Bovet, D. P., & Cesati, M. (2005).** *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.
3. **Coffman, E. G., Elphick, M., & Shoshani, A. (1971).** *System Deadlocks.* ACM Computing Surveys, 3(2), 67-78.
4. **Dennard, R. H., et al. (1974).** *Design of ion-implanted MOSFET's with very small physical dimensions.* IEEE Journal of Solid-State Circuits, 9(5), 256-268.
5. **Hennessy, J. L., & Patterson, D. A. (2017).** *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
6. **Love, R. (2010).** *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.
7. **Moore, G. E. (1965).** *Cramming more components onto integrated circuits.* Electronics, 38(8), 114-117.
8. **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).** *Operating System Concepts* (10th ed.). Wiley.
9. **Stallings, W. (2018).** *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
10. **Sutter, H. (2005).** *The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software.* Dr. Dobb's Journal, 30(3), 16-22.
11. **Tanenbaum, A. S., & Bos, H. (2015).** *Modern Operating Systems* (4th ed.). Pearson.